

1 KNN

1.1 The Very Basics

Learning: find a model f that maps from inputs to outputs given a labeled training dataset.

Inference: use a model f to predict an output y given a new example $\vec{x}$

A model is a function f mapping input $\vec{x}$ to output prediction y .

A model family is a set of models that share the same structure but differ in parameters (e.g. layers in a neural network)

Accuracy is calculated as follows:

$$\frac{\sum_{i=1}^N \mathbb{I}[t^{(i)} = y^{(i)}]}{N}$$

A parameter is the internal variables of a model learned from data whilst training.

A hyperparameter are the settings chosen before training that controls how the learning process works.

Train fit params; **Val** tune hyperparams / pick model; **Test** use once $\rightarrow$ unbiased gen. error (tune on test $\rightarrow$ leakage, optimistic).

Reg. vs class. decided by *output* only (cont. vs discrete). t =target, y =pred; $x_j^{(i)}$: ex. i , feat. j . 100% train acc $\nrightarrow$ good test; May overfit. Data augmentation: changing the training data by a little bit to create new training data (shift/rotate).

Parametric models fix the parameter count. Non-parametric models don't. $\mathcal{O}(1)$ space complexity with respect to the feature count, and never $\Omega(N)$. Hyperparameters are out of this scope.

1.2 KNN

For 1NN: Find $t^{(c)}$ that corresponds to:

$$\vec{x}^{(c)} = \arg \min_{\vec{x}^{(i)} \in \text{training set}} \text{distance}(\vec{x}^{(i)}, \vec{x})$$

For kNN, for the top k other datapoints closest to $\mathbf{x}$, choose the majority class.

Distances ($\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$):

$$L_2: \sqrt{\sum_j (a_j - b_j)^2} \quad \left| \begin{array}{l} \text{Cos: } \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2} \\ L_p: (\sum_j |a_j - b_j|^p)^{1/p} \quad \left| \begin{array}{l} L_\infty: \max_j |a_j - b_j| \end{array} \right. \end{array} \right.$$

Normalize features to mean = 0 and var = 1:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

μ_j, σ_j from training set only; Reuse for val/test (else leakage).

Pros: simple; No training; Works for reg. and class.; Adapts easily to new data.

Cons: curse of dimensionality: in high dims most points are far apart from each other (L_∞ on $[0, 1]^d$: frac. within ϵ of origin = ϵ^d ; $\therefore \epsilon < 1$, $d \uparrow \Rightarrow \epsilon^d \rightarrow 0$); Sensitive to ranges of features (larger range dominates distance calculations); Slow prediction: distance to all N pts, $\mathcal{O}(ND)$ per query; Must store entire training set, $\mathcal{O}(ND)$ space.

Choose k

$k \uparrow \rightarrow$ boundary smoother, less sensitive to individual pts $\rightarrow$ underfit; $k = N \rightarrow$ always majority class.

$k = 1 \rightarrow$ sharp, overfit; 100% train acc (unless dup. $\mathbf{x}$ w/ diff. labels). No single best k , depends on N ; Heuristic $k < \sqrt{N}$ ($k \rightarrow \infty$, $k/N \rightarrow 0$). Pick k (any hyperparam) via val set or cross-val.

Workflow: (1) Split into train/val/test; (2)

For each K , train KNN on train; (3) Compute acc of each on val; (4) Select K w/ best val acc; (5) Compute acc of that K on test.

Cos: direction > magnitude (text). Curse: need exp. more data; Nearest $\approx$ farthest dist.; Large k doesn't fix.

2 Decision Trees

Large trees overfit, small trees underfit. For inference, trace the tree.

Entropy

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$$

Conditional Entropy

$$H(X | Y=y) = -\sum_x p(x | y) \log_2 p(x | y)$$

Expected conditional entropy

$$H(X | Y) = \sum_y p(y) H(X | Y=y)$$

Information gain

$$IG(X | Y) = H(X) - H(X | Y)$$

Properties: $H(X) \geq 0$; $H(Y | X) \leq H(Y)$ (i.e. $IG \geq 0$); Y, X are independent $\Rightarrow IG(Y | X) = 0$ (i.e. $H(Y | X) = H(Y)$); Y fully determined by $X \Rightarrow H(Y | X) = 0 \Rightarrow IG(Y | X) = H(Y)$; Certain $\rightarrow H = 0$; Binary max $H = 1$ bit at $p = .5$.

2.1 Decision Tree Algorithm

Keep splitting until:

- All examples have the same label.
- When there are no features left.
- When a depth limit is reached. (hyperparameter)
- When the loss stops decreasing enough. (hyperparameter)
- Node has < min #samples. (hyperparameter)

Split: pick max IG (not train error $\therefore$ ignores child uncertainty). Leaf $\rightarrow$ majority class. Greedy $\rightarrow$ not globally optimal ($\therefore$ NP-hard). Boundaries axis-aligned ($x_j \leq \theta$). Continuous feat.: only $\leq N-1$ thresholds (midpts between sorted values). Scale-invariant.

3 Linear / GD

Loss function measures how off once training example is.

Pred.: $y = \mathbf{w}^T \mathbf{x} + b$, $\mathbf{y} = X\mathbf{w}$ ($X: N \times (D+1)$, row = example).

Closed form: $\nabla = 0 \Rightarrow X^T X \mathbf{w} = X^T \mathbf{t} \Rightarrow \mathbf{w} = (X^T X)^{-1} X^T \mathbf{t}$.

$\nabla =$ steepest ascent, $\perp$ contours $\rightarrow$ step along $-\nabla$. α too big $\rightarrow$ overshoot/diverge; Too small $\rightarrow$ slow.

Feature map $\psi(\mathbf{x})$ (e.g. $[1, x, x^2]$) $\rightarrow$ nonlinear fit, still linear in $\mathbf{w}$.

Reg. $\mathcal{E} + \lambda \mathcal{R}$: λ small $\rightarrow$ overfit; Big $\rightarrow$ underfit. Cost functions sum up all.

$$\mathcal{L}(y, t) = \frac{1}{2}(y - t)^2$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y^{(i)}, t^{(i)})$$

For the linear model $\mathbf{y} = W\mathbf{x}$, the **cost** function is:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)})^2$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} (X\mathbf{w} - \mathbf{t})^T (X\mathbf{w} - \mathbf{t})$$

3.1 Gradient of the Cost Function

Setup: $N = \#$ examples, $D = \#$ input features.

Dummy $x_0^{(i)} = 1 \Rightarrow b = w_0 x_0^{(i)}$:

$$\mathbf{x}^{(i)} = [1, x_1^{(i)}, \dots, x_D^{(i)}]^T \in \mathbb{R}^{(D+1) \times 1}$$

$$\mathbf{w} = [w_0, \dots, w_D]^T \in \mathbb{R}^{(D+1) \times 1}$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} \in \mathbb{R}$$

$$X = \begin{bmatrix} (\mathbf{x}^{(1)})^T \\ \vdots \\ (\mathbf{x}^{(N)})^T \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \quad (\text{row} = \text{example})$$

ple)

$$\mathbf{t}, \mathbf{y} \in \mathbb{R}^{N \times 1}, \mathbf{y} = X\mathbf{w}$$

Cost: loss = single example; Cost = average over a set of examples.

$$\mathcal{L}(y^{(i)}, t^{(i)}) = \frac{1}{2}(y^{(i)} - t^{(i)})^2 \in \mathbb{R}$$

$$\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_i \mathcal{L} \in \mathbb{R}$$

$$\text{Let } \mathbf{e} = \mathbf{y} - \mathbf{t} \in \mathbb{R}^{N \times 1}: \mathbf{e}^T \mathbf{e} = \sum_i (y^{(i)} - t^{(i)})^2$$

$$\Rightarrow \mathcal{E} = \frac{1}{2N} \mathbf{e}^T \mathbf{e} = \frac{1}{2N} (X\mathbf{w} - \mathbf{t})^T (X\mathbf{w} - \mathbf{t}) = \frac{1}{2N} \|X\mathbf{w} - \mathbf{t}\|_2^2$$

Gradient:

$$\frac{\partial \mathcal{L}}{\partial y^{(i)}} = y^{(i)} - t^{(i)}, \quad \frac{\partial y^{(i)}}{\partial w_j} = x_j^{(i)}$$

$$\Rightarrow \frac{\partial \mathcal{L}}{\partial w_j} = (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\Rightarrow \frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\Rightarrow \nabla \mathbf{w} \mathcal{L} = (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)} \in \mathbb{R}^{D+1}$$

$$\Rightarrow \nabla \mathbf{w} \mathcal{E} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

$$\text{Let } r^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)} \in \mathbb{R}, \mathbf{r} = X\mathbf{w} - \mathbf{t} \in \mathbb{R}^{N \times 1};$$

$$\nabla \mathbf{w} \mathcal{E} = \frac{1}{N} \sum_i r^{(i)} \mathbf{x}^{(i)} = \frac{1}{N} X^T \mathbf{r} = \frac{1}{N} X^T (X\mathbf{w} - \mathbf{t}) \in \mathbb{R}^{D+1}$$

KEY: $\sum_i r^{(i)} \mathbf{x}^{(i)} = X^T \mathbf{r}$

($\therefore$ columns of X^T are $\mathbf{x}^{(i)}$, weighted by $r^{(i)}$)

Solve:

$$(1) \text{ Direct: } \nabla \mathcal{E} = 0 \Rightarrow X^T X \mathbf{w} = X^T \mathbf{t}$$

$$\Rightarrow \mathbf{w} = (X^T X)^{-1} X^T \mathbf{t}$$

Pro: closed form, exact & provably optimal, no iteration.

Con: CFS may not exist for most problems; Can't generalize to most models; Computationally expensive; $(X^T X)^{-1}$ is $\mathcal{O}(D^3)$.

$$(2) \text{ GD: } \underbrace{\mathbf{w}}_{(D+1) \times 1} \leftarrow \mathbf{w} - \underbrace{\frac{\alpha}{N} X^T (X\mathbf{w} - \mathbf{t})}_{(D+1) \times 1}$$

The regularizer:

$$\frac{\lambda}{2} \sum_{j=0}^D w_j^2$$

The gradient of the regularizer:

$$\frac{\partial}{\partial w_j} \left(\frac{\lambda}{2} \sum_{j=0}^D w_j^2 \right) = \lambda w_j$$

$$\nabla \mathbf{w} \left(\frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \right) = \lambda \mathbf{w}$$

In the event the bias term is left in:

$$\mathcal{E}(\mathbf{w}) = \frac{1}{2N} (X\mathbf{w} + b - \mathbf{t})^T (X\mathbf{w} + b - \mathbf{t})$$

$$\nabla \mathbf{w} \mathcal{E}(\mathbf{w}) = \frac{1}{N} X^T (X\mathbf{w} + b - \mathbf{t})$$

$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)}) x_j^{(i)}$$

$$\frac{\partial \mathcal{E}(\mathbf{w})}{\partial b} = \frac{1}{N} (X\mathbf{w} + b - \mathbf{t})$$

3.2 Stochastic GD

Batch GD is what GD normally does. It can be slow.

Stochastic GD just randomly picks a training

example. It can be very noisy. Where i at random,

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$$

Mini-batch GD falls in-between.

Batch size can be tuned as a hyperparameter

Iteration: 1 weight update from 1 mini-batch of B . **Epoch:** every example used once ($= N/B$ iterations).

	Batch	Mini-batch	SGD
#ex/update	all N	$B, 1 < B < N$	1
Acc of grad	exact	noisy ($\downarrow$ as $B \uparrow$)	noisiest
Cost/update	$O(N)$	$O(B)$	$O(1)$
Vectorize	high (mem?)	high, fit hw	low

Mini-batch grad = unbiased est.; SGD noise $\rightarrow$ escape plateaus (no global-opt guarantee). Ravine: very diff. grad magnitudes per dim $\rightarrow$ oscillate; Fix: normalize features.

3.3 Vectorizing Tips

Where X is $N \times D$ and all other variables can have their shape inferred:

Dot product

$$\sum_{i=1}^N u_i v_i = \mathbf{u}^T \mathbf{v}$$

Hadamard product

$$\mathbf{u} \odot \mathbf{v} = \text{diag}(\mathbf{u}) \mathbf{v} = \left(\mathbf{u}^T \text{diag}(\mathbf{v}) \right)^T$$

Sum a vector

$$\mathbf{u}^T \mathbf{1} = \sum_{i=1}^N u_i$$

Summing a matrix on axis 1 – horizontal squish $> - <$ (shape N)

$$X \mathbf{1}$$

Summing a matrix on axis 0 – vertical squish $\downarrow$ (shape D)

$$\mathbf{1}^T X$$

This is a column vector, and you may need to transpose it.

Row-wise multiplication

$$\text{diag}(\mathbf{u}) X = \begin{bmatrix} - & u_1 \mathbf{x}^{(1)} & - \\ - & u_i \mathbf{x}^{(i)} & - \\ - & u_N \mathbf{x}^{(N)} & - \end{bmatrix}$$

Column-wise multiplication

$$X \cdot \text{diag}(\mathbf{v}) = \begin{bmatrix} | & | & | \\ v_1 \mathbf{x}_1 & v_2 \mathbf{x}_2 & v_3 \mathbf{x}_3 \\ | & | & | \end{bmatrix}$$

Tip: in a chain of matrix multiplications, the **height** of the leftmost matrix will be the height of the output. E.g. for $X^T \mathbf{r}$, output is $D \times 1$.

$$\sum_{i=1}^N r^{(i)} \mathbf{x}^{(i)} = X^T \mathbf{r}$$

Shape reasoning: the summed index (N) must become the *inner* dimension and cancel; The dimension that appears first in the result goes on the left.

Stack predictions: $(X \mathbf{w})_i = \mathbf{w}^T \mathbf{x}^{(i)}$, so $X \mathbf{w} \in \mathbb{R}^{N \times 1}$ stacks all predictions.

Scalar with vector ($s \in \mathbb{R}$)

$$\alpha_i = u_i + s \Rightarrow \boldsymbol{\alpha} = \mathbf{u} + s \mathbf{1}$$

$$\beta_i = s u_i \Rightarrow \boldsymbol{\beta} = s \mathbf{u}$$

Outer product

$$M_{ij} = u_i v_j \Rightarrow M = \mathbf{u} \mathbf{v}^T \quad (P \times Q)$$

Inner product $\rightarrow$ scalar; Element-wise $\rightarrow$ vector; Outer $\rightarrow$ matrix.

Weighted sum of vectors (M rows $\mathbf{m}_i^T$ vs. columns $\mathbf{m}_j$)

$$\sum_i u_i \mathbf{m}_i = M^T \mathbf{u} \text{ (rows)}$$

$$\sum_j u_j \mathbf{m}_j = M \mathbf{u} \text{ (cols)}$$

Stack of dot products (M rows $\mathbf{m}_i^T$)

$$\alpha_i = \mathbf{m}_i^T \mathbf{u} \Rightarrow \boldsymbol{\alpha} = M \mathbf{u}$$

Two weights per term

$$\sum_i u_i v_i \mathbf{m}_i = M^T (\mathbf{u} \odot \mathbf{v}) = M^T \text{diag}(\mathbf{u}) \mathbf{v}$$

Double sums

$$\sum_i \sum_j u_i v_j m_{ij} = \mathbf{u}^T M \mathbf{v}$$

$$\sum_i \sum_j u_i v_j m_{ij}^2 = \mathbf{u}^T (M \odot M) \mathbf{v}$$

$$\sum_i \sum_k u_i m_{ik} u_k = \mathbf{u}^T M \mathbf{u}$$

Pitfall: $\mathbf{a} \odot \mathbf{b} \neq \mathbf{a} \text{diag}(\mathbf{b})$ — $(P \times 1)(P \times P)$ is undefined; Use $\text{diag}(\mathbf{b}) \mathbf{a}$.

Pitfall: $\frac{1}{N} (X \mathbf{w} - \mathbf{t})^T X$ is a $1 \times (D+1)$ row vector; The gradient must be a column $\Rightarrow \frac{1}{N} X^T (X \mathbf{w} - \mathbf{t})$.

Variants of LR gradient ($\mathbf{r} = X \mathbf{w} - \mathbf{t}$)

- Per-weight λ_j : $\frac{1}{N} X^T \mathbf{r} + \boldsymbol{\lambda} \odot \mathbf{w} = \frac{1}{N} X^T \mathbf{r} + \text{diag}(\boldsymbol{\lambda}) \mathbf{w}$

- Per-example weight α_i : $\sum_i \alpha_i r^{(i)} \mathbf{x}^{(i)} = X^T \text{diag}(\boldsymbol{\alpha}) \mathbf{r}$

- Per-feature scale α_j : predictions $X \text{diag}(\boldsymbol{\alpha}) \mathbf{w}$, gradient gains a $\text{diag}(\boldsymbol{\alpha})$ on the left: $\frac{1}{N} \text{diag}(\boldsymbol{\alpha}) X^T \mathbf{r}'$, $\mathbf{r}' = X \text{diag}(\boldsymbol{\alpha}) \mathbf{w} - \mathbf{t}$

Absorb bias: $\mathbf{w}_1^T \mathbf{x}_1 + b = \mathbf{w}_2^T \mathbf{x}_2$ with $\mathbf{w}_2 = \begin{bmatrix} b \\ \mathbf{w}_1 \end{bmatrix}$, $\mathbf{x}_2 = \begin{bmatrix} 1 \\ \mathbf{x}_1 \end{bmatrix}$ (both $(D+1) \times 1$).

4 Logistic Regression

$y = \sigma(\mathbf{w}^T \mathbf{x}) = p(t = 1 | \mathbf{x})$, threshold 0.5. vs kNN: fixed $\mathbf{w} \rightarrow$ fast $O(D)$ pred.

Linear model is:

$$z = \mathbf{w}^T \mathbf{x}$$

Prediction is:

$$y = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

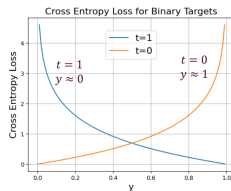
For which the loss functions are:

- Zero-One Loss: $L_{0-1}(y, t) = \mathbb{I}[y \neq t]$
 - Non-differentiable / zero gradients (cannot update weights).
- Squared Error Loss: $L_{SE}(z, t) = \frac{1}{2}(z - t)^2$
 - Penalizes confident correct predictions.
- Sigmoid + Squared Error: $y = \sigma(z) = \frac{1}{1+e^{-z}}$, $L_{SE}(y, t) = \frac{1}{2}(y - t)^2$
 - Suffers from vanishing gradients (when predictions are extremely wrong).

- Cross entropy: $\begin{cases} -\log(y) & t = 1 \\ -\log(1 - y) & t = 0 \end{cases}$

Where cross entropy can be:

$$-t \log(y) - (1 - t) \log(1 - y)$$



4.1 Cross-Entropy Loss Equations

Linear model

$$z = \mathbf{w}^T \mathbf{x}$$

Classification

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z) (1 - \sigma(z))$$

$$\frac{\partial y}{\partial z} = y(1 - y)$$

Loss

$$\mathcal{L}_{CE}(y, t) = -t \log(y) - (1 - t) \log(1 - y)$$

Gradient Descent Update

Where $\mathcal{E}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{CE}(y^{(i)}, t^{(i)})$

$$\begin{aligned} \frac{\partial \mathcal{L}_{CE}}{\partial w_j} &= \frac{\partial \mathcal{L}_{CE}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j} \\ &= \left(-\frac{t}{y} + \frac{1-t}{1-y} \right) \cdot y(1-y) \cdot x_j \\ &= (y - t) x_j \end{aligned}$$

Tip: re-use variables, don't sub all!

The gradient descent update for logistic regression is:

$$\begin{aligned} w_j &\leftarrow w_j - \alpha \frac{\partial \mathcal{E}}{\partial w_j} \\ &= w_j - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)} \end{aligned}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} X^T (\sigma(X \mathbf{w}) - \mathbf{t})$$

CE: $\mathcal{L} \rightarrow \infty$ for confident wrong; $\partial \mathcal{L} / \partial z = y - t$ (no saturation); Convex in $\mathbf{w}$; Max-likelihood. Sigmoid+SE: $\partial \mathcal{L} / \partial z = (y - t)y(1 - y) \approx 0$ when very wrong.

Boundary: $y = 0.5 \iff z = \mathbf{w}^T \mathbf{x} = 0$ (hyperplane); Class 1 where $z \geq 0$.

4.2 Softmax Regression ($K \geq 3$)

Target one-hot $\mathbf{t} \in \{0, 1\}^K$ (1 at true class). Not integers: imply order + distance, renumbering changes model; One-hot: no order, all pairs equidistant.

Logits $\mathbf{z} = W \mathbf{x} \in \mathbb{R}^K$, $W \in \mathbb{R}^{K \times (D+1)}$, row $k = [b_k, \mathbf{w}_k^T]$, $z_k = \mathbf{w}_k^T \mathbf{x}$ (unnormalized). Predict $\arg \max_k z_k$.

$$y_k = \frac{\exp(z_k)}{\sum_{k'=1}^K \exp(z_{k'})}, \quad \mathbf{y} = \text{softmax}(\mathbf{z}) \in \mathbb{R}^K$$

Properties: (1) prob. dist.: $y_k \in (0, 1)$, $\sum_k y_k = 1$. (2) order-preserving ($\cdot: \exp \uparrow$) $\Rightarrow \arg \max y = \arg \max z$, softmax doesn't change boundary; y_k = confidence. (3) smooth, differentiable approx. of $\arg \max \Rightarrow$ GD works. (4) $K = 2$: $y_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z_1 - z_2)$, $y_2 = \sigma(-z)$.

Loss $\mathcal{L}_{CE} = -\sum_k t_k \log y_k = -\mathbf{t}^T \log \mathbf{y} = -\log y_{\text{true}}$ (1 nonzero term).

Grad $\frac{\partial \mathcal{L}}{\partial z_k} = y_k - t_k$, $\nabla_{\mathbf{w}_k} \mathcal{L} = (y_k - t_k) \mathbf{x}$

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \frac{\alpha}{N} \sum_i (y_k^{(i)} - t_k^{(i)}) \mathbf{x}^{(i)}$$

Matrix: $\nabla_W \mathcal{E} = \frac{1}{N} (Y - T)^T X$ ($Y, T \in \mathbb{R}^{N \times K}$, rows $\mathbf{y}^{(i)T}, \mathbf{t}^{(i)T}$).

Boundary i/j needs BOTH: (1) $z_i = z_j \iff (\mathbf{w}_i - \mathbf{w}_j)^T \mathbf{x} = 0$ (line); (2) $z_k \leq z_i \forall k \neq i, j$.

Steps: solve (1) for line $\rightarrow$ substitute into (2) $\rightarrow$ inequality gives ray/segment $\rightarrow$ all 3 lines meet where $z_1 = z_2 = z_3 \rightarrow$ test pt per region. At most $K(K-1)/2$ boundaries.

Linear boundaries, convex regions; Can't do nonlinear.